SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2,BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 10 **Total Marks:100**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I **NAWAB SAAD (192472266)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: NAWAB SAAD

Q21. Closest Pair – Negative and Positive Coordinates

Given the set of points {(-2,-1), (3,4), (0,0), (5,1), (-1,-2)}, apply the Closest Pair Brute Force algorithm. Clearly interpret the problem and justify the chosen method. Compute distances between all possible pairs and present the calculations clearly. Identify the closest pair and count the number of pairwise comparisons. Analyze the time complexity of the brute force approach. Verify the correctness of the result and interpret its relevance in spatial analysis.

Aim:

To find the closest pair of points from the given set using the **Brute Force Closest Pair Algorithm**.

Problem Statement:

Given the set of points:

$$\{(-, 2, -1), , (3, 4), , (0, 0)\ (5, 1)\ (-, 1, -2)\}$$

Find the closest pair by:

- Calculating the Euclidean distance between every possible pair.
- Identifying the minimum distance.
- Counting the total number of comparisons.
- Analyzing the time complexity.

Algorithm (Brute Force):

1. Input all points.
2. Set the minimum distance as infinity.
3. Compare every point with every other point.

4. Calculate Euclidean distance:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

5. If the calculated distance is smaller than the current minimum, update the minimum distance.
6. Continue until all pairs are compared.
7. Display the closest pair and minimum distance.

Given Points:

Points	Coordinates
P1	(-2,-1)
P2	(3,4)
P3	(0,0)
P4	(5,1)
P5	(-1,-2)

Total number of points:

$$n = 5$$

Pairwise Distance Calculations:

1. P1 and P2:

$$= \sqrt{(3+2)^2 + (4+1)^2}$$
$$= \sqrt{25+25} = \sqrt{50} = 7.07$$

2. P1 and P3:

$$= \sqrt{(0+2)^2 + (0+1)^2}$$
$$= \sqrt{4+1} = \sqrt{5} = 2.24$$

3. P1 and P4:

$$= \sqrt{(5+2)^2 + (1+1)^2}$$
$$= \sqrt{49+4} = \sqrt{53} = 7.28$$

4. P1 and P5:

$$= \sqrt{(-1+2)^2 + (-2+1)^2}$$
$$= \sqrt{1+1} = \sqrt{2} = 1.41$$

5. P2 and P3:

$$= \sqrt{(0-3)^2 + (0-4)^2}$$
$$= \sqrt{9} + \sqrt{16} = \sqrt{25} = 5.00$$

6. P2 and P4:

$$= \sqrt{(5-3)^2 + (1-4)^2}$$
$$= \sqrt{4} + 9 = \sqrt{13} = 3.61$$

7. P2 and P5:

$$= \sqrt{(-1-3)^2 + (-2-4)^2}$$
$$= \sqrt{16+36} = \sqrt{52} = 7.21$$

8. P3 and P4:

$$= \sqrt{(5-0)^2 + (1-0)^2}$$
$$= \sqrt{25+1} = \sqrt{26} = 5.10$$

9. P3 and P5:

$$= \sqrt{(-1-0)^2 + (-2-0)^2}$$
$$= \sqrt{1+4} = \sqrt{5} = 2.24$$

10. P4 and P5:

$$= \sqrt{(-1-5)^2 + (-2-1)^2}$$
$$= \sqrt{36+9} = \sqrt{45} = 6.71$$

Summary Table:

Pair	Distance
P1 – P2	7.07
P1 – P3	2.24
P1 – P4	7.28
P1 – P5	1.41
P2 – P3	5
P2 – P4	3.61
P2 – P5	7.21
P3 – P4	5.1
P3 – P5	2.24
P4 – P5	6.71

Closest Pair:

Closest Points:

$(-, 2-1)$ and $(-, 1-2)$

Minimum Distance:

$\boxed{\sqrt{2} \approx 1.41}$

Number of Pairwise Comparisons:

The brute force algorithm compares every unique pair.

Formula:

$$\frac{n(n-1)}{2}$$

For $n = 5$:

$$\frac{5 \times 4}{2} = 10$$

Total Comparisons = 10

Time Complexity Analysis:

Case	Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Since every pair must be checked, the running time is quadratic regardless of the input.

Correctness Verification:

- All **10 unique pairs** were evaluated exactly once.
- The smallest computed distance is **1.41**.
- No other pair has a distance less than **1.41**.

Therefore, the closest pair is correctly identified as:

$$\boxed{(-, 2 - 1) \text{ and } (-, 1 - 2)}$$

Relevance in Spatial Analysis

The Closest Pair problem is widely used in spatial applications to identify the nearest two locations. It is useful in:

- Geographic Information Systems (GIS)
- GPS navigation
- Robotics and path planning
- Collision detection in computer graphics
- Wireless sensor networks
- Drone and autonomous vehicle navigation

The brute force method is simple and guarantees the correct answer, making it suitable for small datasets. However, for large datasets its $O(n^2)$time complexity becomes inefficient, and divide-and-conquer algorithms with $O(n \log n)$complexity are preferred.

PROGRAM :

```
import math

# List of points

points = [(-2, -1), (3, 4), (0, 0), (5, 1), (-1, -2)]

min_distance = float('inf')

closest_pair = ()

comparisons = 0

print("Pairwise Distance Calculations:\n")


# Compare every pair of points
```

```
for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]
        # Euclidean Distance
        distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
        comparisons += 1
        print(f"{points[i]}  <-->  {points[j]} = {distance:.2f}")
        # Update minimum distance
        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])
print("\n-----------------------------------")
print("Closest Pair :", closest_pair)
print("Minimum Distance :", round(min_distance, 2))
print("Total Comparisons :", comparisons)
print("-----------------------------------")
# Time Complexity
print("\nTime Complexity Analysis")
print("Best Case    : O(n^2)")
print("Average Case : O(n^2)")
print("Worst Case   : O(n^2)")
```

OUTPUT:

CO2AT1.py - C:\Users\Nawab Saad\CO2AT1.py (3.13.14)

File Edit Format Run Options Window Help

```
import math

# List of points
points = [(-2, -1), (3, 4), (0, 0), (5, 1), (-1, -2)]

min_distance = float('inf')
closest_pair = ()
comparisons = 0

print("Pairwise Distance Calculations:\n")

# Compare every pair of points
for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]

        # Euclidean Distance
        distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

        comparisons += 1

        print(f"{points[i]}  <-->  {points[j]} = {distance:.2f}")

        # Update minimum distance
        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])

print("\n-----------------------------------")
print("Closest Pair :", closest_pair)
print("Minimum Distance :", round(min_distance, 2))
print("Total Comparisons :", comparisons)
print("-----------------------------------")

# Time Complexity
print("\nTime Complexity Analysis")
print("Best Case    : O(n^2)")
print("Average Case : O(n^2)")
print("Worst Case   : O(n^2)")
```

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:0
 (AMD64)] on win32
Enter "help" below or click "Help" above for more inform
>>>
========== RESTART: C:\Users\Nawab Saad\CO2AT1
Pairwise Distance Calculations:

(-2, -1)  <-->  (3, 4) = 7.07
(-2, -1)  <-->  (0, 0) = 2.24
(-2, -1)  <-->  (5, 1) = 7.28
(-2, -1)  <-->  (-1, -2) = 1.41
(3, 4)  <-->  (0, 0) = 5.00
(3, 4)  <-->  (5, 1) = 3.61
(3, 4)  <-->  (-1, -2) = 7.21
(0, 0)  <-->  (5, 1) = 5.10
(0, 0)  <-->  (-1, -2) = 2.24
(5, 1)  <-->  (-1, -2) = 6.71

-----------------------------------
Closest Pair : ((-2, -1), (-1, -2))
Minimum Distance : 1.41
Total Comparisons : 10
-----------------------------------

Time Complexity Analysis
Best Case    : O(n^2)
Average Case : O(n^2)
Worst Case   : O(n^2)
>>>
```

Result

The **Closest Pair Brute Force Algorithm** was successfully applied to the given points. After evaluating **10 pairwise distances**, the closest pair was found to be:

$$\boxed{(-, 2 - 1) \text{ and } (-, 1 - 2)}$$

with a minimum Euclidean distance of

$$\boxed{\sqrt{2} \approx 1.41}$$

The algorithm has a time complexity of $O(n^2)$and is correct for this dataset.